

CISC 3667: Game Design and Development

In-Class Activity 6: Build a JavaScript Mini-Game

Assigned: Thursday, October 8

Submission: Brightspace by 11:59 PM (Individual Submission)

Overview

Build a small game in a **new, blank** [CodePen](#) using HTML, CSS, and JavaScript. Part 1 walks you through the systems every game needs: a canvas, a player, keyboard input, movement, and a game loop. **Write the code in your own CodePen as you follow the steps.** Then choose **one** of the three game options in Part 2. You will design and code its objects, rules, challenge, feedback, and ending yourself. You may consult the course notes and examples, but **do not fork or copy a completed example game.**

Part 1: Follow the Starter Tutorial

Step 1: Create the page

Create a new CodePen. In its **HTML** panel, add a canvas, a place to display the game status, and a Restart button:

```
<h2>My JavaScript Mini-Game</h2>
<p id="instructions">Use the arrow keys or WASD to move.</p>
<p id="status">Game ready</p>
<canvas id="gameCanvas" width="600" height="400"></canvas>
<button id="restartButton" type="button"> Restart </button>
```

You may change the title and instructions when you decide what your game will be.

Step 2: Style the game

In the **CSS** panel, give the page a readable layout. Make the canvas visible by adding a background or border. For example:

```
body {  
  font-family: Arial, sans-serif;  
  text-align: center;  
}  
  
#gameCanvas {  
  display: block;  
  max-width: 100%;  
  margin: 20px auto;  
  border: 3px solid #333;  
  background: #e8f2fa;  
}
```

You may change the colors and layout. Keep the game status and instructions easy to find.

Step 3: Create the player and draw the game

In the **JavaScript** panel, get the canvas and its drawing context. Create a player object with a position, size, and speed:

```
const canvas = document.getElementById("gameCanvas");  
const context = canvas.getContext("2d");
```

```
const statusDisplay = document.getElementById("status");
const restartButton = document.getElementById("restartButton");

const player = {
  x: 30,
  y: 30,
  size: 24,
  speed: 180
};

const keys = {};
let gameOver = false;
let lastTime = null;
```

Write a **draw()** function that fills the background and draws the player as a colored rectangle. Call it once to make sure the player appears before adding movement.

```
function draw() {
  // Cover the previous frame with the background color.
  context.fillStyle = "#e8f2fa";
  context.fillRect(0, 0, canvas.width, canvas.height);

  // Draw the player at its current position.
  context.fillStyle = "#2563eb";
  context.fillRect(player.x, player.y, player.size, player.size);
}

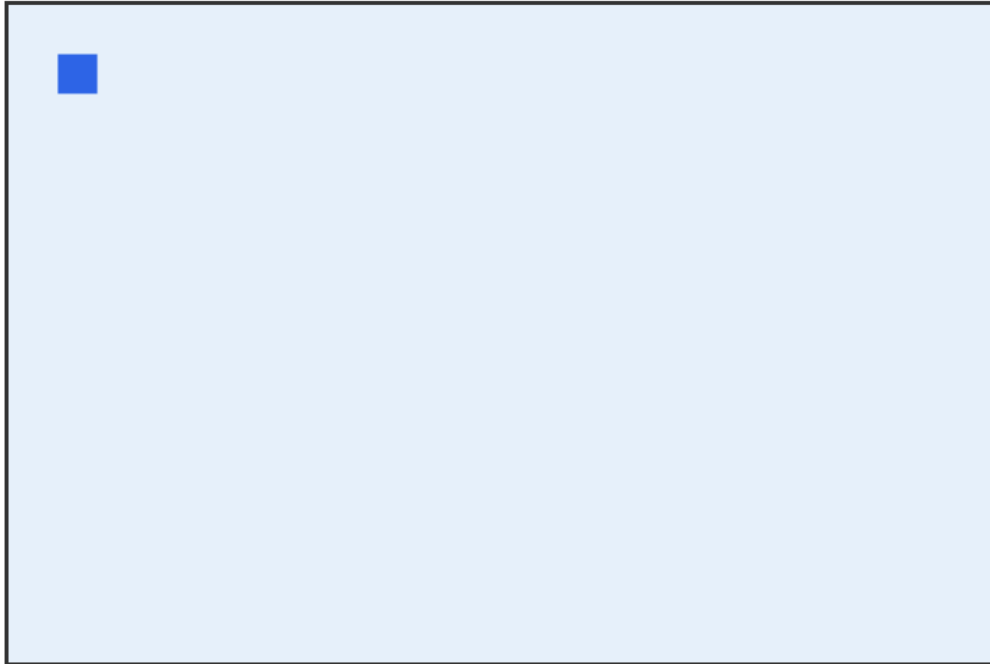
draw();
```

At this point, you should see a blue square near the top-left corner. If you do not, check your HTML IDs and look for an error in the CodePen console before continuing.

My JavaScript Mini-Game

Use the arrow keys or WASD to move.

Game ready



Restart

Step 4: Add keyboard movement

First, add event listeners below your **draw()** function. The **keys** object records whether each key is currently pressed:

```
window.addEventListener("keydown", function(event) {  
  const key = event.key.toLowerCase();  
  keys[key] = true;
```

```

    // Keep arrow keys from scrolling the page.
    if (key === "arrowup" || key === "arrowdown" || key === "arrowleft" ||
        key === "arrowright") {
        event.preventDefault();
    }
});

window.addEventListener("keyup", function(event) {
    const key = event.key.toLowerCase();
    keys[key] = false;
});

```

Now add an **update(deltaTime)** function. It reads the recorded keys, moves the player, and keeps the player inside the canvas:

```

function update(deltaTime) {
    if (gameOver) {
        return;
    }

    let moveX = 0;
    let moveY = 0;

    if (keys.arrowleft || keys.a) {
        moveX -= 1;
    }
}

```

```
if (keys.arrowright || keys.d) {
    moveX += 1;
}

if (keys.arrowup || keys.w) {
    moveY -= 1;
}

if (keys.arrowdown || keys.s) {
    moveY += 1;
}

// Keep diagonal movement from being faster.
const movementLength = Math.hypot(moveX, moveY);
if (movementLength > 0) {
    moveX /= movementLength;
    moveY /= movementLength;
}

// Speed is measured in pixels per second.
player.x += moveX * player.speed * deltaTime;
player.y += moveY * player.speed * deltaTime;

// Keep the entire player inside the canvas.
player.x = Math.max(0, Math.min(canvas.width - player.size, player.x));
player.y = Math.max(0, Math.min(canvas.height - player.size, player.y));
}
```

The **update()** function is ready, but you will not see movement yet. It needs a game loop to call it repeatedly.

Step 5: Add a Game Loop

Your **update(deltaTime)** function contains the movement rules, but it does not run automatically. The game needs to call **update()** and **draw()** repeatedly so that movement appears continuous. This repeated process is called the **game loop**.

Add this function below **update()**:

```
function gameLoop(currentTime) {  
    // On the first frame, there is no previous time to compare.  
    if (lastTime === null) {  
        lastTime = currentTime;  
    }  
    // Find the time since the previous frame.  
    // Divide by 1000 to convert milliseconds to seconds.  
    const deltaTime = Math.min((currentTime - lastTime) / 1000, 0.05);  
    // Save this frame's time for the next frame.  
    lastTime = currentTime;  
    // Change the game state, then display it.  
    update(deltaTime);  
    draw();  
    // Ask the browser to run the loop again.  
    requestAnimationFrame(gameLoop);  
}
```

At the bottom of your JavaScript, **replace the standalone `draw()` ; call from Step 3** with:

```
requestAnimationFrame(gameLoop);
```

What Happens During Each Frame?

The **game loop** repeats three steps:

1. **Update:** Move the player, check interactions, and apply the game's rules.
2. **Draw:** Show the current state of the game.
3. **Repeat:** Ask the browser to run the loop again.

Delta time tells the game how much time has passed since the previous update. This helps movement and timers stay consistent. The code also limits unusually large time jumps, such as when a player returns to a paused tab.

Start the loop with **`requestAnimationFrame(gameLoop)`** at the bottom of your JavaScript. The loop schedules the following frames itself.

Test before continuing: Make sure the player moves with the arrow keys or WASD and stops at the canvas edges. In Part 2, you will add your chosen game's rules to **`update()`** and its objects to **`draw()`**.

Part 2: Choose One Game

Choose **one** of the following options. Each game must have a clear goal, at least one challenge, visible feedback, a win or loss outcome, and a working Restart button.

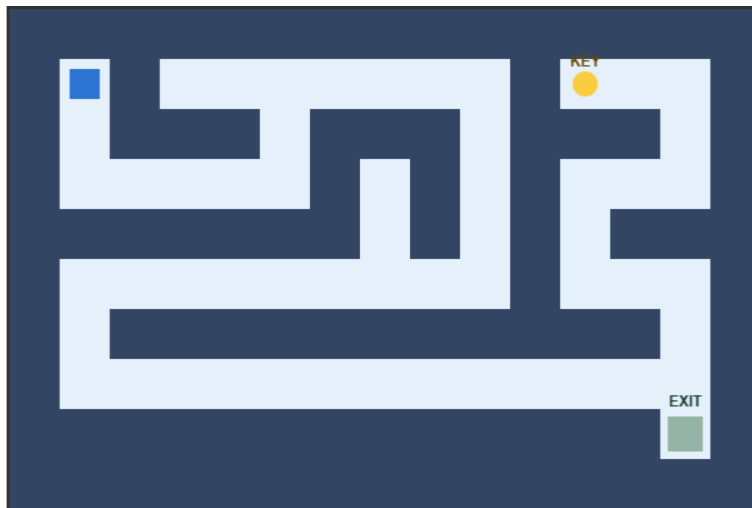
Option A: Maze Escape

Create a small maze using rectangular walls and an exit area.

Maze Escape

Find the yellow key, then reach the green exit. Move with arrow keys or WASD.

Time: 41 | Find the key!



- Draw at least **five walls** that block the player.
- Place a **key** and a visible exit. The player must collect the key before reaching the exit to win.
- The player loses if the **45-second timer** runs out.
- Show the key status and a clear result. Restart must reset the game.

Development hint: Store the walls in an array and check whether the player's next position would overlap a wall before allowing the move. Track whether the key has been collected with a Boolean variable. When the player reaches the exit, check that variable before ending the game.

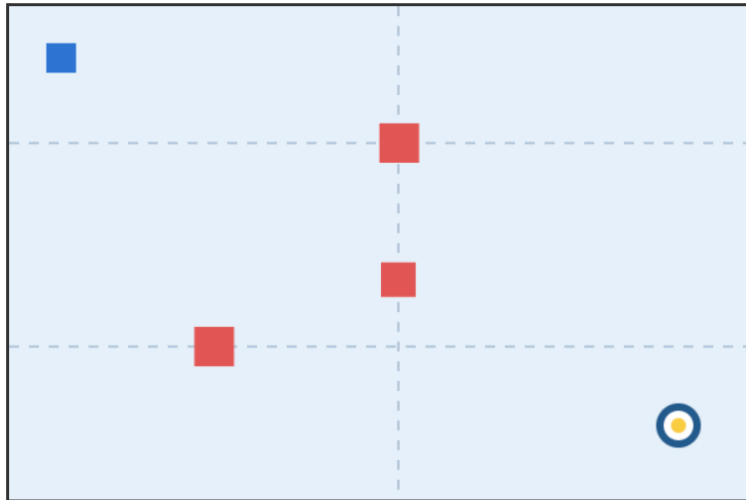
Option B: Dodge and Survive

Create a game in which the player must reach a **bullseye** while avoiding at least one moving hazard.

Dodge and Survive

Reach the bullseye before time runs out. Avoid hazards. Move with arrow keys or WASD.

Time: 26 | Reach the bullseye!



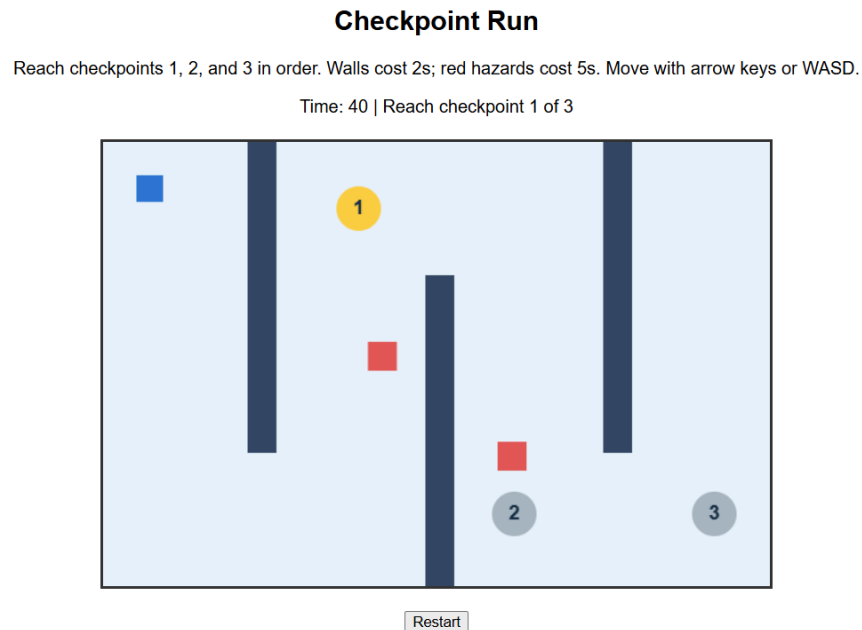
Restart

- Draw a visible bullseye for the player to reach.
- Add at least **one hazard** that moves automatically.
- The player **wins** by touching the bullseye.
- The player **loses** if a hazard touches them or if the **30-second timer** runs out.
- Display the time remaining and show a clear win or loss message.
- Make the hazard's movement predictable enough for the player to react. For example, it could move back and forth and reverse direction at an edge.
- Restart must return the player, hazard, bullseye, and timer to their starting states.

Development hint: Give the hazard a position, size, speed, and direction. Update its position each frame and reverse its direction when it reaches an edge. Check whether the player overlaps the hazard or touches the bullseye.

Option C: Checkpoint Run

Guide the player through **three numbered checkpoints in order** before the 45-second timer runs out.



- Show which checkpoint the player needs next. Touching a later checkpoint early does not count.
- Add at least **two walls** that block movement and deduct time when touched.
- Add at least **one moving hazard** that deducts time and sends the player back to the start.
- The player wins after checkpoint 3 and loses if time runs out.
- Show a clear result and provide a Restart button.

Development hint: Store the next required checkpoint in a variable. Check collisions with walls and hazards separately because they have different effects. When the player reaches the correct checkpoint, advance the variable and update the status display. Use a short cooldown for collision penalties so one contact does not deduct time repeatedly.

Part 3: Finish the Game

Whichever option you choose, your game must include:

- Instructions explaining the controls and goal.
- A visible player and clearly distinguishable game objects.
- A game state that changes as the player acts.
- Collision or contact rules appropriate to your option.
- Feedback showing progress or time remaining.
- A clear **win** outcome and a clear **loss** outcome.
- A Restart button that resets positions, timers, progress, and the end-of-game state.

You may use simple shapes. Artwork, sound, multiple levels, and advanced menus are not required.

Part 4: Test and Revise

Ask yourself:

1. Can a new player understand the goal and controls without reading your code?
2. Do collisions behave as expected near the edges of objects?
3. Can the player reach the goal?
4. Does the loss condition work?
5. Does Restart return the game to a fresh, playable state?
6. Is the challenge fair? What did you change after testing?

Test the win and loss conditions rather than assuming they work. If a feature remains incomplete, leave your attempt in the CodePen and explain the issue in your reflection.

Submission

Each student must submit their own work to Brightspace by the end of the day. Submit:

1. A link to your **saved CodePen**. Check that the link opens your game and its code.
2. A short reflection of approximately **150–250 words** answering:
 - Which game option did you choose? What is the player's goal?
 - What objects and state values does your game track?
 - How does your game detect contact or collisions?
 - What did you change after testing?
 - Did you use an AI tool? If so, identify it and explain how you used it.

You may use AI to explain code, learn a programming concept, or debug an error. Follow the course AI policy: do not use AI for creative brainstorming or assets, and document any AI assistance.